

Von Pygame zu Ursina

Kurze Lernanleitung für PixelPortalRift – Phase 1

Für den ersten eigenen Umbau eines kleinen Spiels

Erstellt auf Grundlage des beigefügten Projekts

Inhalt

1	1. Was soll am Ende funktionieren?	1
	1.1 Dein Projekt als Ausgangspunkt	1
2	2. Vorbereitung	1
3	3. Der kleinste Ursina-Start	1
	3.1 Eine sichtbare Figur	2
	3.2 Einen einfachen Boden bauen	2
4	4. Bewegung: Pygame übersetzen	3
	4.1 Eingaben als eigene Funktion	3
5	5. Der erste vollständige Umbau	3
6	6. Was wird später übertragen?	4
	6.1 Übersetzungsregel	5
7	7. Aufgaben für den Schüler	5
8	Quellen und weiterführende Seiten	5

1. Was soll am Ende funktionieren?

Du hast in Phase 1 bereits die Grundidee von **PixelPortalRift** kennengelernt: Es gibt ein Fenster, eine Spielfigur, eine Welt aus Blöcken und eine Schleife, die das Spiel immer wieder aktualisiert. Für den ersten Umbau musst du noch nicht das ganze Spiel mit Inventar, Portalen und Gegnern nachbauen.

Das Ziel dieser Übung ist kleiner:

1. Ein Ursina-Fenster öffnet sich.
2. Einige Würfel bilden einen einfachen Boden.
3. Eine Figur steht in der Welt.
4. Die Figur kann sich mit W, A, S und D bewegen.
5. Ein Block wird als eigene Klasse erzeugt.

> **Merksatz:** Wir übertragen zuerst die *Idee* und die *Struktur*, nicht jede einzelne Pygame-Zeile.

1.1 Dein Projekt als Ausgangspunkt

Im vorhandenen Projekt ist `main.py` der Startpunkt. Dort wird die Klasse `Game` erzeugt. Die Methode `run()` enthält die Hauptschleife. Die Dateien in `utils/` teilen die Aufgaben auf: `player.py` kümmert sich um den Spieler, `world_gen.py` um die Welt und `constants.py` um feste Werte.

| Pygame-Projekt | Bedeutung | Erste Ursina-Idee | | — | — | — | | `pygame.display.set_mode(...)` | Fenster öffnen | `app = Ursina()` | | `while self.running:` | Hauptschleife | `app.run()` | | `pygame.event.get()` | Tastatur- und Mausereignisse | `input(key)` | | `self.update(dt)` | Spielzustand verändern | `update()` | | `pygame.draw...` oder Bilder | Etwas sichtbar zeichnen | `Entity(...)` | | Player als eigene Klasse | Spieler verwalten | `class Player(Entity)` |

Die offiziellen Dokumentationen beschreiben Pygame als Bibliothek, bei der man die Schleife selbst kontrolliert [1]. In Ursina übernimmt die Engine diese Schleife; sichtbare Dinge werden meist als `Entity` angelegt [2].

2. Vorbereitung

Installiere Ursina in derselben Python-Umgebung, in der du dein neues Beispiel ausführen möchtest:

```
pip install ursina
```

Lege im Projektordner eine neue Datei `ursina_phase1.py` an. Am besten lässt du das ursprüngliche Pygame-Projekt unverändert. So kannst du jederzeit vergleichen.

Starte das Beispiel später mit:

```
python ursina_phase1.py
```

Wenn ein Fehler erscheint, lies zuerst die letzte Zeile der Fehlermeldung. Häufige Ursachen sind ein fehlendes Paket, ein falscher Dateiname oder eine falsche Einrückung.

3. Der kleinste Ursina-Start

Schreibe zuerst nur diesen kurzen Code. Er soll ein Fenster öffnen und wieder schließen können.

```
from ursina import Ursina

app = Ursina()

app.run()
```

Ursina() erstellt die Anwendung. app.run() startet die von Ursina verwaltete Hauptschleife. Das ist der größte Unterschied zum bisherigen Pygame-Einstieg: Du musst nicht selbst while running schreiben.

3.1 Eine sichtbare Figur

Jetzt kommt ein Würfel dazu. In Ursina ist ein sichtbares Objekt normalerweise eine Entity. Ein Würfel ist ein Modell mit dem Namen cube.

```
from ursina import Ursina, Entity, color

app = Ursina()

player = Entity(
    model="cube",
    color=color.azure,
    scale=(1, 2, 1),
    position=(0, 1, 0),
)

app.run()
```

Wichtige Eigenschaften sind model, color, scale und position. Die Ursina-Dokumentation beschreibt eine Entity als ein Objekt mit Position, Drehung, Größe, Modell und Farbe [2].

3.2 Einen einfachen Boden bauen

Mehrere Würfel können in einer Schleife erzeugt werden. So entsteht ein Boden, ohne dass du jeden Würfel einzeln schreiben musst.

```
for x in range(-5, 6):
    for z in range(-5, 6):
        Entity(
            model="cube",
            color=color.green,
            position=(x, 0, z),
        )
```

Die beiden Schleifen bedeuten: Für jede X-Position werden mehrere Z-Positionen ausprobiert. Die dritte Koordinate, y, beschreibt die Höhe. In diesem 3D-Beispiel ist der Boden deshalb bei y=0.

4 4. Bewegung: Pygame übersetzen

In Pygame fragst du oft in jedem Durchlauf ab, ob eine Taste gehalten wird. In Ursina kannst du dafür eine Funktion `update()` verwenden. `time.dt` steht für die Zeit seit dem letzten Bild. Dadurch bleibt die Bewegung bei unterschiedlichen Bildraten gleichmäßig.

```
from ursina import Ursina, Entity, color, held_keys, time

app = Ursina()

player = Entity(
    model="cube",
    color=color.azure,
    scale=(1, 2, 1),
    position=(0, 1, 0),
)

def update():
    player.x += held_keys["d"] * 5 * time.dt
    player.x -= held_keys["a"] * 5 * time.dt
    player.z += held_keys["w"] * 5 * time.dt
    player.z -= held_keys["s"] * 5 * time.dt

app.run()
```

Dabei ist `held_keys["d"]` normalerweise 1, wenn D gehalten wird, und sonst 0. Bei A ziehen wir Bewegung ab. Das ist die gleiche Grundidee wie in Pygame, aber Ursina stellt die Eingabe bequemer bereit.

4.1 Eingaben als eigene Funktion

Für einzelne Aktionen, zum Beispiel Springen oder ein Menü, ist `input(key)` übersichtlich:

```
def input(key):
    if key == "space":
        print("Sprung!")
```

Bei jedem Tastendruck erhält die Funktion den Namen der Taste. Zum Lernen reicht es, zunächst eine Nachricht auszugeben. Erst danach wird daraus eine echte Sprungbewegung.

5 5. Der erste vollständige Umbau

Nun kannst du alles in einer Datei zusammenführen. Dieses Beispiel ist bewusst klein gehalten. Es benutzt noch keine Texturen, Kollisionen, Portale oder Speichersysteme.

```
from ursina import Ursina, Entity, color, held_keys, time
```

```

app = Ursina()

class Block(Entity):
    def __init__(self, position=(0, 0, 0), block_color=color.green):
        super().__init__(
            model="cube",
            color=block_color,
            position=position,
            collider="box",
        )

player = Entity(
    model="cube",
    color=color.azure,
    scale=(1, 2, 1),
    position=(0, 1, 0),
)

for x in range(-5, 6):
    for z in range(-5, 6):
        Block(position=(x, 0, z))

def update():
    player.x += held_keys["d"] * 5 * time.dt
    player.x -= held_keys["a"] * 5 * time.dt
    player.z += held_keys["w"] * 5 * time.dt
    player.z -= held_keys["s"] * 5 * time.dt

def input(key):
    if key == "space":
        print("Sprung vorbereitet")

app.run()

```

Block ist hier die erste kleine Übertragung aus der Weltlogik. Später könnte die Klasse Eigenschaften wie Blockname, Härte oder Textur bekommen. Im vorhandenen Pygame-Projekt liegen vergleichbare Informationen in BLOCK_PROPERTIES in `utils/constants.py`.

6 6. Was wird später übertragen?

Versuche die Funktionen in kleinen Schritten zu ergänzen. Nach jeder Änderung soll das Programm noch starten.

| Nächster Schritt | Was du untersuchen kannst | Ursina-Werkzeug | | – | – | – | | Kamera | Wie folgt die Kamera dem Spieler? | `camera.position` | | Kollision | Wie verhindert man, dass der Spieler durch Blöcke läuft? | `collider`, `raycast()` | | Texturen | Wie wird ein Blockbild verwendet? | `texture="dateiname"` | | HUD | Wie zeigt man Koordinaten an? | Text | | Mausklick | Wie erkennt man einen angeklickten Block? | `collider`, `on_click` | | Weltenwechsel | Wie werden mehrere Szenen verwaltet? | eigene Weltklasse, Entity-Gruppen |

Ursina stellt für Entities unter anderem Kollisionen, Mausreaktionen und eine automatisch aufgerufene `update()`-Methode bereit [2]. Du musst deshalb nicht jede Pygame-Funktion eins zu eins nachbauen.

6.1 Übersetzungsregel

Wenn du eine Stelle aus Pygame umschreiben möchtest, stelle dir drei Fragen:

1. *Was ist es?* Ein Objekt, eine Eingabe, eine Berechnung oder eine Darstellung?
2. *Wo gehört es hin?* In eine Klasse, in `update()`, in `input(key)` oder in eine kleine Hilfsfunktion?
3. *Was macht Ursina schon für mich?* Fenster, Schleife und viele Eingaben müssen nicht vollständig selbst programmiert werden.

7 7. Aufgaben für den Schüler

Aufgabe A – Farben ändern. Ändere die Farbe des Spielers und des Bodens. Teste danach, ob das Spiel weiterhin startet.

Aufgabe B – Größere Welt. Ändere `range(-5, 6)` so, dass die Welt größer oder kleiner wird. Beobachte, wie viele Blöcke entstehen.

Aufgabe C – Bewegung untersuchen. Ändere die Zahl 5 in `update()`. Wann bewegt sich die Figur langsam, wann schnell?

Aufgabe D – Eigener Block. Erzeuge neben dem Boden einen roten Block. Verwende dafür `Block(position=(2, 1, 2), block_color=color.red)`.

Aufgabe E – Rückblick. Markiere in deinem Code die Stellen, die in Pygame ungefähr `draw`, `event.get`, `key.get_pressed` und `clock.tick` entsprochen hätten.

> *Erfolgskriterium für Phase 1:* Das Fenster startet, der Boden wird sichtbar, die Figur ist sichtbar und bewegt sich mit den vier Tasten. Mehr ist für den ersten Versuch nicht nötig.

8 Quellen und weiterführende Seiten

Die Anleitung verwendet die offizielle Pygame-Dokumentation zur Spielschleife und die offizielle Ursina-Dokumentation zu Entities, Eingaben, `update()` und Kollisionen.

[1 *Pygame Documentation*, Quick start und Referenz, abgerufen am 4. September 2026.]

[2 *Ursina Entity Basics*, offizielle Dokumentation zu Entities, Modellen, Position, `update()` und Eingaben, abgerufen am 4. September 2026.]

[3 *Ursina API Reference*, offizielle API-Referenz zu Ursina, Entity und `app.run()`, abgerufen am 4. September 2026.]